

SIMATS ENGINEERING
 Department of Computer Science and Engineering
ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA12	Course Title:	Computer Architecture
CO Assessed:	CO2 – Precision (BL3)	Assessment:	Analytical Problem Solving
Weightage:	45%	Total Marks:	100
CO2 Statement:	Apply integer and floating-point arithmetic algorithms including Booth's multiplication, restoring/non-restoring division, and IEEE 754 standard operations to solve fixed-point and floating-point computational problems.		
Student Name:	S. Haleshwar Praya	Reg. No.:	192524284
Section / Batch:	A14DS	Date:	3/8/26

Instructions to Students

- Identify the appropriate arithmetic algorithm or number representation required for the given computational problem.
- Analyse the step-by-step execution of integer and floating-point arithmetic operations using the selected algorithm.
- Apply Booth's multiplication, restoring/non-restoring division, and IEEE 754 standards to obtain accurate results.
- Compute binary multiplication, division, normalization, and floating-point arithmetic using appropriate algorithms.
- Interpret the computed results by evaluating their accuracy, precision, and suitability for the given application.

QUESTIONS

1. A digital computer performs signed integer arithmetic using 8-bit 2's complement representation. Consider the numbers -75 and $+38$. Analyze how the processor performs both addition and subtraction by converting the operands to binary, executing the operations step-by-step, detecting overflow (if any), and interpreting the final results in both binary and decimal forms.
2. A high-performance processor employs an 8-bit carry look-ahead adder (CLA) to enhance arithmetic speed. Given two binary operands, analyse how the generate and propagate signals are produced, how carry bits are calculated simultaneously, and compare the execution time and hardware complexity of both adders. Explain why CLA is widely used in modern processors.
3. A digital processing unit executes signed multiplication using Booth's algorithm. Multiply $+13$ and -6 , and analyze the algorithm step-by-step by showing the contents of the accumulator, multiplier, multiplicand, and Booth bit after each iteration. Explain the purpose of arithmetic right shifts and discuss how Booth's algorithm reduces the number of addition and subtraction operations.
4. A binary arithmetic unit supports both restoring and non-restoring division methods for signed integer operations. Analyze the division of the binary numbers $27 \div 5$ using each algorithm by showing the intermediate remainder values, quotient formation, and register contents after every iteration. Compare the two methods based on execution steps, computational complexity, and hardware efficiency, and justify which algorithm is more suitable for high-speed processors.
5. A floating-point processing unit represents real numbers using the IEEE 754 standard. Analyze the representation of the decimal number -42.375 in both single-precision (32-bit) and double-precision (64-bit) formats by identifying the sign, exponent, and mantissa fields. Further, explain how floating-point addition is performed between -42.375 and $+15.125$, including exponent alignment, mantissa operations, normalization, rounding, and the impact of precision on the final result.

Code of Conduct

I S. Haleshwar Praya (Name / Reg No. 192524284) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the Student: S. Praya

Question 1

Given

number 1 = -75

number 2 = +38

Convert decimal numbers into 8 bit binary -

+75

$$75_{10} = 01001011_2$$

-75 (2's complement)

$$\begin{array}{r} 10110100 \quad (1's \text{ complement}) \\ \hline 10110101 \quad (\text{add } 1) \end{array}$$

$$-75 = 10110101$$

+38

$$38_{10} = 00100110_2$$

(a) Addition (-75) + (+38)

$$\begin{array}{r} 10110101 \\ 00100110 \\ \hline 11011011 \end{array}$$

MSB = 1 (-ve)

again 2's complement

$$\begin{array}{r} 11011011 \quad (\text{Result}) \\ 00100100 \quad (1's \text{ complement}) \\ \hline 00100101 \quad (\text{add } 1) \end{array}$$

Result = -37

→ Operands have opposite signs, no overflow

• Binary result: 11011011

Decimal result: -37

(B) subtraction: $(-75) - (+38)$

+38: 00100110

Is complement = 110011001
1
11011010

-38: 11011010

now add:-

10110101 (-75)
11011010 (-38)
①10001111

ignore the carry

Result = 10001111

Result = -113

01110000
1
01110001 =

Both operands are negative, and the result is also

Binary result: 10001111

Decimal result: -113

10

011

11

Operation - 2

Given operands

Let

$$A = 10110101$$

$$B = 00100110$$

Generate (G) and propagate (P):

$$G_i = A_i \times B_i$$

$$P_i = A_i \oplus B_i$$

Bit	A_i	B_i	G_i	P_i
0	1	0	0	1
1	0	1	0	1
2	1	1	1	0
3	0	0	0	0
4	1	0	0	1
5	1	1	1	0
6	0	0	0	0
7	1	0	0	1

Carry Equations

$$C_0 = 0$$

$$C_1 = G_0 + P_0 C_0$$

$$C_2 = G_1 + P_1 C_1$$

$$C_3 = G_2 + P_2 C_2$$

$$C_4 = G_3 + P_3 C_3$$

$$C_5 = G_4 + P_4 C_4$$

$$C_6 = G_5 + P_5 C_5$$

$$C_7 = G_6 + P_6 C_6$$

$$C_8 = G_7 + P_7 C_7$$

Sum Equation:

$$S_i = P_i \oplus G_i$$

using the carry values, the final sum is produced in one stage instead of waiting for carries to ripple through

why CLA used in modern processors?

→ Carry bits are generated simultaneously

→ Performs high-speed arithmetic operations

Comparison of CLA and Ripple Carry adder:-

feature	Ripple carry adder	Carry look ahead adder.
Carry generation	one by one	simultaneously
speed	slow	very fast
Delay	high	low
Hardware	simple	more complex
Cost.	low	higher

Question - 3

Given

* multiplicand (M) = +13

* multiplier (Q) = -6

Use 5 bit 2's complement representation

* +13 = 01101

* -13 = 10011

* -6 = 11010

(2's complement of 13)

Initial values;

* A (accumulator) = 00000

* Q (multiplier) = 11010

* M (multiplicand) = 01101

* $M = 10011$

* Q_{-1} (Booth bit) = 0

* number of bits = 5

Booth's Rules

$Q_0 Q_{-1}$

operation

00

No operation

01

$A = A + M$

10

$A = A - M$

11

No operation

Iteration 1

Initial

$A = 00000$

$Q_{-1} = 0$

$Q = 11010$

$Q_0 Q_{-1} = 00$ (no operation)

Arithmetic right shift:

$A = 00000$

$Q = 01101$

$Q_{-1} = 0$

Iteration - 2

$A = 00000$

$Q_{-1} = 0$

$Q = 01101$

$Q_0 Q_{-1} = 10$

$[A = A - M]$

$$\begin{array}{r} 000000 \\ + 00011 \\ \hline 10011 \end{array}$$

ARS: $A = 11001$

$Q = 10110$

$Q_{-1} = 1$

Iteration - 3

$A = 11001$

$Q_{-1} = 1$

$[A = A + M]$

$Q = 10110$

$Q_0 Q_{-1} = 01$

$$\begin{array}{r}
 1100 \\
 01101 \\
 \hline
 00110
 \end{array}$$

ARS: $A = 00011$

$Q = 01011$

$Q_{-1} = 0$

Iteration - 4

$A = 00011$

$Q = 01011$

$Q_{-1} = 0$

$Q_0 Q_{-1} = 10$

$[A = A + M]$

$A = 11011$

$Q = 00101$

$Q_{-1} = 1$

$$\begin{array}{r}
 00011 \\
 10011 \\
 \hline
 10110
 \end{array}$$

Iteration - 5

$A = 11011$

$Q_{-1} = 1$

$Q = 00101$

$Q_0 Q_{-1} = 11$

[no operation]

ARS: $A = 11101$

$Q = 10010$

$Q_{-1} = 1$

Combine A and Q:

$A = 11101$

$Q = 10010$

$$11101100101_2$$

This is -78_{10} in 10 bit 2's complement

Since,

$$13 \times (-6) = -78$$

Hence, the multiplication is correct

Ripple Algorithm reduces addition and subtraction operations:-

- Checks Q_0 and Q_{-1} together
- Performs addition/subtraction only for 01 and 10 transitions
- Reduces operations for consecutive 1's in the multiplier

Question - 4

Given

$$\text{dividend} = 27 = 11011_2$$

$$\text{divisor} = 5 = 00101_2$$

$$\text{no. of bits} = 5$$

$$A \text{ (Accumulator / Remainder)} = 00000$$

$$Q \text{ (Quotient)} = 11011$$

$$M \text{ (divisor)} = 00101$$

Restoring division algorithm:

Iteration 1

Left shift (A/Q)

$$A = 00001$$

Iteration 2

Left shift

$$A = 00011$$

Iteration 3

Left shift

$$A = 00111$$

$$Q = 11$$

$$00001 - 00101 = 11100 (-ve)$$

$$00011 - 00101 = 11110 (-ve)$$

Restore :-

$$11100 + 00101 = 00001$$

Restore:

$$11110 + 00101 = 00011$$

$$Q_0 = 0$$

$$Q_0 = 1$$

Iteration 4

left shift

$$A = 00011$$

$$Q = 10010$$

$$A = A - M$$

$$00011 - 00101 = 11110 (-ve)$$

Restore .

$$11110 + 00101 = 00011$$

$$Q_0 = 0$$

$$\text{Quotient} = 00101_2 = 5$$

$$\text{Remainder} = 00010_2 = 2$$

Non Restoring division Algorithm:-

Iteration 1

left shift

$$A = 00001$$

subtract M

$$00001 - 00101 = 11100$$

Iteration 2

left shift

Since A is -ve
and M

$$11001 + 00101 = 11110$$

Iteration 3

left shift

add M

$$11101 + 00101 = 00010$$

negative
(p < 0.1)

negative
(p < 0.1)

negative
(p < 0.1)

Stochastic

left diff

sublabel 1

00101 - 00101

00000

110110011

Stochastic

left diff

sublabel 1

00001 - 00101 - 11000

negative (p < 0.1)

11001 - 00101 - 00000

Stochastic - 00101 - 1

Stochastic - 00010 - 0

Non isolating division is not suitable for high speed

Reassembles hardware:

-> if avoids unnecessary modifications

-> if performs known arithmetic operations

-> if executes faster

-> if requires simpler and less efficient hardware

-> if improves the overall performance of the

Question - 5

IEEE 754 Representation of 42.375

Convert decimal to Binary.

$$42_{10} = 101010_2$$

Fraction Part

$$0.375 \times 2 = 0.75 \rightarrow 0$$

$$0.75 \times 2 = 1.5 \rightarrow 1$$

$$0.5 \times 2 = 1.0 \rightarrow 1$$

So,

$$0.375 = 0.011_2$$

Therefore,

$$42.375 = 101010.011_2$$

Hence

$$-42.375 = -101010.011_2$$

Normalize

$$101010.011_2 = 1.01010011 \times 2^5$$

(A) IEEE 754 single Precision (32 bit)

$$\text{Sign bit (s)} = 1 \text{ (-ve. no)}$$

$$\text{Exponent} = 5 + 127 = 132$$

$$132_{10} = 10000100_2$$

$$\text{Mantissa} = 0101001100000000000000$$

Final 32 bit representation

Sign

1

Exponent

10000100

Mantissa

0101001100000000000000

LD
LIFE
P

Sign bit = 1

Exponent bias = 1023

$$\text{Exponent} = 5 + 1023 = 1028$$

$$1028_{10} = 100\ 000\ 00100_2$$

mantissa = 01010011000000000000000000000000
00000000000000000000

Floating Point Addition

Add:

$$-42 \cdot 375 + 15 \cdot 125$$

Step 1: Convert to Binary

$$\therefore 42.375 = -1.01010011 \times 2^5$$

$$\bullet 15.125 = +1.111001 \times 2^3$$

Exponent Alignment:-

larger exponent = 5

Shift the mantissa of 15.125 right by 2 bits
 1.111001×2^3

$$1 \cdot 111001 \times 2^3$$



$$0.01111001 \times 2^5$$

now both have exponent 15

mantissa operation:-

Since the signs are different,

Subtract,

$$1.01010011 - 0.01111001 = 0.11011010 \text{ (-ve)}$$

Final result:

$$-27.25$$

$$\text{Binary: } -1.101101 \times 2^4$$

$$\text{Decimal: } -27.25$$

Steps in floating point addition:-

- Convert the numbers to IEEE 754 format.
- Align the exponents by shifting the smaller mantissa.
- Add or subtract the mantissa and normalize the result.
- Round and store the final sign, exponent and mantissa.

Impact of Precision:-

- Single Precision uses a 23 bit mantissa.
- Double Precision uses a 52 bit mantissa.
- Double Precision gives higher accuracy than single Precision.
- Double Precision produces smaller rounding error.